

Applied Bayesian Statistics in Animal & Biological Sciences

2) (Hierarchical) linear prediction models

Henk J. van Lingen

hvanling@uoguelph.ca

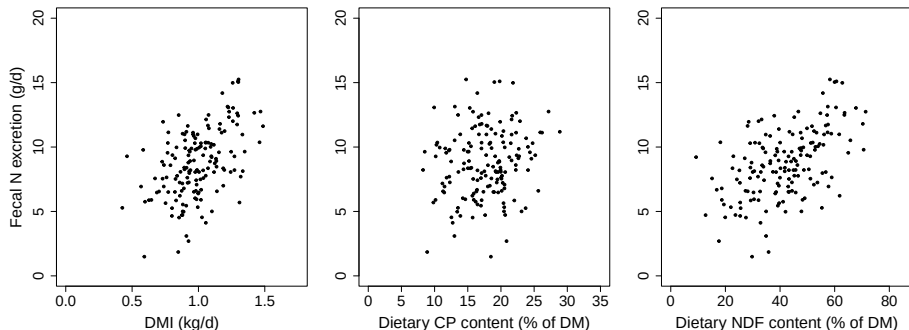
May 11, 2026



Today's program

- Prior distributions for linear models
- Writing "fixed-effects" model STAN code and evaluate simulation output
- Assessing RSTAN output
- Writing "mixed-effects" or hierarchical model STAN code and evaluate simulation output
- Consider vectorization in STAN code

Predicting fecal nitrogen excretion from sheep



$$y_i = \beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \beta_3 x_{3,i} + \epsilon_i \quad (1)$$

$$\epsilon_i \sim N(0, \sigma_e^2) \quad (2)$$

Apply Bayes' theorem to linear models

For parameters that follow continuous distributions:

$$\underbrace{p(\boldsymbol{\theta}|\mathbf{y})}_{\text{posterior}} \stackrel{\text{Bayes' rule}}{=} \frac{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\mathbf{y})}{p(\mathbf{y})} \propto \underbrace{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\mathbf{y})}_{\text{prior times likelihood}} \quad (3)$$

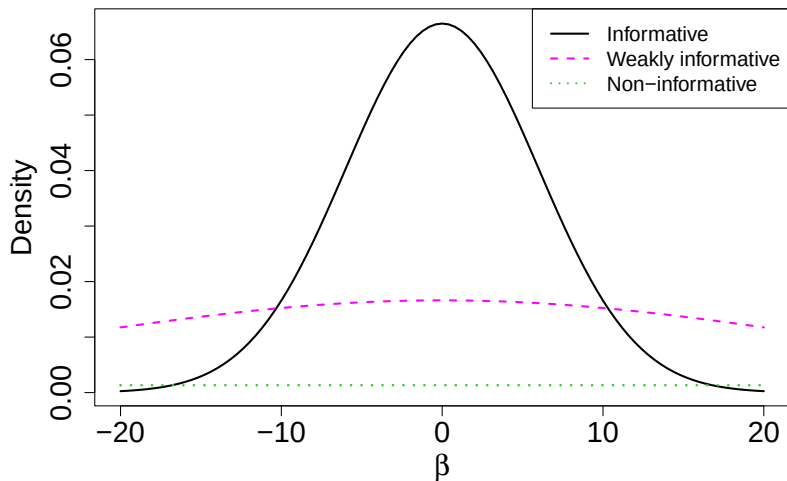
Apply Bayes' theorem to linear models

For parameters that follow continuous distributions:

$$\underbrace{p(\boldsymbol{\theta}|\mathbf{y})}_{\text{posterior}} = \underbrace{\frac{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\mathbf{y})}{p(\mathbf{y})}}_{\text{Bayes' rule}} \propto \underbrace{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\mathbf{y})}_{\text{prior times likelihood}} \quad (3)$$

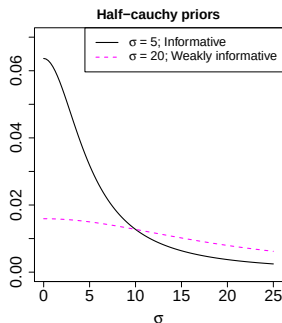
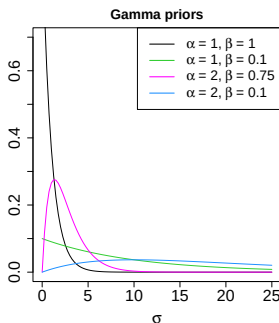
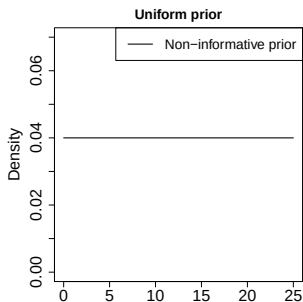
- Parameters are considered random variables assigned to a distribution
- Obtain posterior knowledge through exploring $p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\mathbf{y})$, i.e. use data and prior knowledge
- Powerful Markov-chain Monte Carlo algorithms are run for n iterations from which we sample

Priors for linear prediction model - β parameters



$$\beta_0, \beta_1, \beta_2, \beta_3 \sim \{N(0, 6), N(0, 24), N(0, 10000)\} \quad (4)$$

Priors for linear prediction model - σ parameter



$$\sigma_e \sim \text{Uniform}(0, 25); \text{ also called a flat prior} \quad (5)$$

$$\sigma_e \sim \text{Gamma}(\alpha, \beta); \text{ with } \alpha, \beta \text{ the shape and rate parameters} \quad (6)$$

$$\sigma_e \sim \text{half-cauchy}(\mu = 0, \sigma); \text{ defined on } [\mu, \infty) \quad (7)$$

with μ, σ the location and scale parameters

"Fixed-effects" linear model code in STAN

```
FN_FixedEf <- "  
data {  
  int<lower=0> N;           // number of data points  
  real y[N];               // the Fecal nitrogen data  
  real x_DMI[N];           // the DMI data  
  real x_CP[N];            // the CP data  
  real x_NDF[N];           // the NDF data  
} // End of data  
  
parameters{  
  real<lower=0> sigma_e;    // residual std dev  
  real beta_0;             // regression coefficients  
  real beta[3];            // regression coefficients  
} // End of parameters  
  
model {  
  for(i in 1:N){  
    y[i] ~ normal(beta_0 + beta[1]*x_DMI[i] + beta[2]*x_CP[i] + beta[3]*x_NDF[i], sigma_e);  
  } // End of for loop i  
  beta_0 ~ normal(0,300);  
  beta ~ normal(0,300);  
  sigma_e ~ cauchy(0,15);  
} // End of model
```

- STAN is C++ based, note the semi-colons...
- STAN only takes integers and numeric values, and uses σ , not σ^2

RSTAN code to perform model simulation

```
### RSTAN code ###
library(rstan)
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

### Initialize STAN code in R
FN_FixedEf_mod <- stan_model(model_code=FN_FixedEf,model_name="FN_FixedEf")

### Generate data input for STAN model as a list
n_sample <- 160; n_studies <- 10

set.seed(6708)
sheep_FN_dat <- data.frame(x_DMI = rnorm(n_sample, 0.977, 0.209), # DMI
                           x_CP = rnorm(n_sample, 17.7, 4.33),    # CP
                           x_NDF = rnorm(n_sample, 41.6, 13.3),   # NDF
                           s_s = rep(rnorm(n_studies, 0, sqrt(1.68)), each=n_sample/n_studies),
                           e_res = rnorm(n_sample, 0, sqrt(2.25)) )

sheep_FN_dat <- data.frame(sheep_FN_dat, Study=rep(1:n_studies, each=n_sample/n_studies),
                           y = -4.36 + 6.03*sheep_FN_dat$x_DMI + 0.109*sheep_FN_dat$x_CP +
                               0.115*sheep_FN_dat$x_NDF + sheep_FN_dat$s_s + sheep_FN_dat$e_res)

sheep_FN_dat <- list(N=n_sample, y=sheep_FN_dat$y,
                    x_DMI=sheep_FN_dat$x_DMI, x_CP=sheep_FN_dat$x_CP, x_NDF=sheep_FN_dat$x_NDF)
```

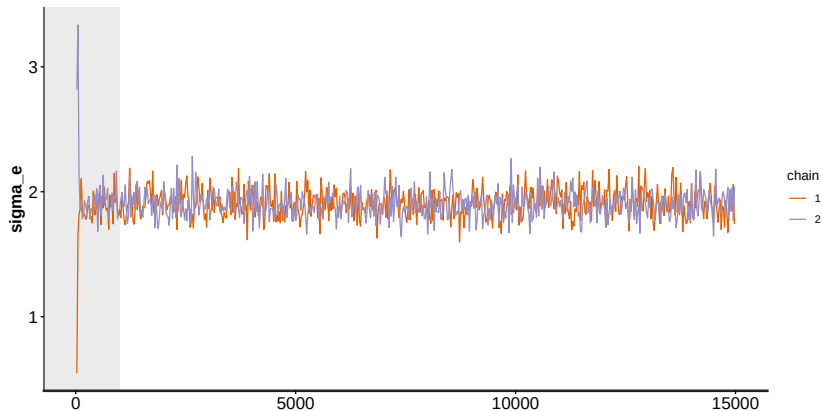
RSTAN code to perform simulation - save output

```
### Run the HMC simulation of the STAN model
sheep.rs1 <- sampling(FN_FixedEf_mod, data=sheep_FN_dat, pars=c("beta_0","beta","sigma_e"),
                     iter=15e3, warmup=1e3, thin=25, chains=2)

### Write output to csv file
write.csv(summary(sheep.rs1, pars=c("beta_0","beta","sigma_e"))$summary, "sheep_FN_FixedEf_out.csv")

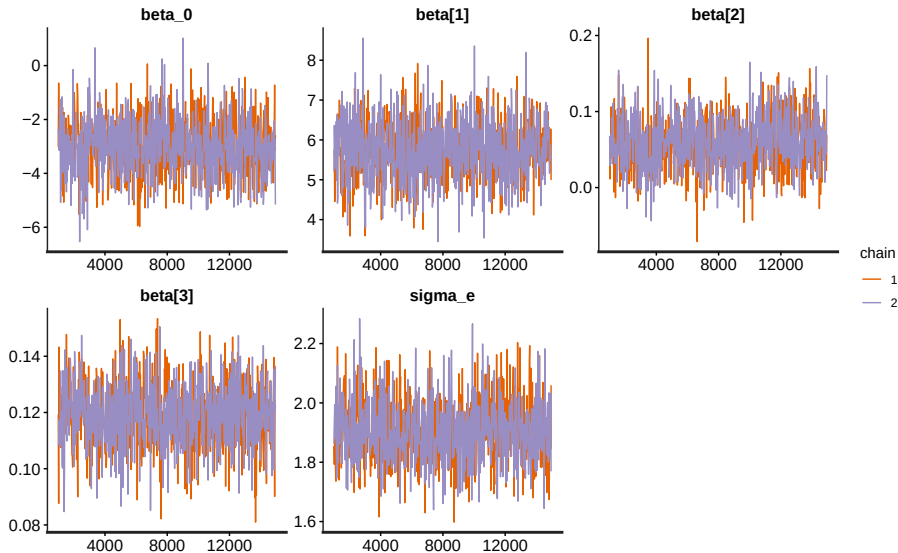
### Save traceplot in pdf format
pdf("ModPars_vs_iterations.pdf", height=7, width=11)
traceplot(sheep.rs1, pars=c("beta_0","beta","sigma_e"))
dev.off()
```

Model simulation in RSTAN - Traceplot



- 2 Markov-chain Monte Carlo chains run for 15×10^3 iterations
- First few iterations (burnin or warmup; here $n = 1000$) are discarded to let the chains converge

Model simulation in RSTAN - Traceplot



Evaluate (R)STAN model output (posteriors)

```
print(summary(sheep.rs1, pars=c("beta_0", "beta", "sigma_e"))$summary)
```

##	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
## beta_0	-3.246	0.035	1.088	-5.320	-3.946	-3.306	-2.534	-1.085	944.527	0.999
## beta[1]	5.721	0.024	0.747	4.288	5.236	5.728	6.220	7.231	959.127	1.003
## beta[2]	0.120	0.001	0.036	0.048	0.096	0.119	0.144	0.191	1118.421	1.000
## beta[3]	0.100	0.000	0.011	0.078	0.093	0.100	0.108	0.121	953.045	0.999
## sigma_e	1.902	0.003	0.107	1.710	1.828	1.897	1.972	2.126	1031.951	0.999

- Output comprises mean, se of mean, sd and various quantiles of parameter values

Evaluate (R)STAN model output (posteriors)

```
print(summary(sheep.rs1, pars=c("beta_0", "beta", "sigma_e"))$summary)
```

##	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
## beta_0	-3.246	0.035	1.088	-5.320	-3.946	-3.306	-2.534	-1.085	944.527	0.999
## beta[1]	5.721	0.024	0.747	4.288	5.236	5.728	6.220	7.231	959.127	1.003
## beta[2]	0.120	0.001	0.036	0.048	0.096	0.119	0.144	0.191	1118.421	1.000
## beta[3]	0.100	0.000	0.011	0.078	0.093	0.100	0.108	0.121	953.045	0.999
## sigma_e	1.902	0.003	0.107	1.710	1.828	1.897	1.972	2.126	1031.951	0.999

- Output comprises mean, se of mean, sd and various quantiles of parameter values
- Notice the effective samples sizes (`n_eff`):

$$2 \cdot \frac{15 \times 10^3 - 1 \times 10^3}{25} = 1.12 \times 10^3$$

Evaluate (R)STAN model output (posteriors)

```
print(summary(sheep.rs1, pars=c("beta_0", "beta", "sigma_e"))$summary)
```

##		mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
##	beta_0	-3.246	0.035	1.088	-5.320	-3.946	-3.306	-2.534	-1.085	944.527	0.999
##	beta[1]	5.721	0.024	0.747	4.288	5.236	5.728	6.220	7.231	959.127	1.003
##	beta[2]	0.120	0.001	0.036	0.048	0.096	0.119	0.144	0.191	1118.421	1.000
##	beta[3]	0.100	0.000	0.011	0.078	0.093	0.100	0.108	0.121	953.045	0.999
##	sigma_e	1.902	0.003	0.107	1.710	1.828	1.897	1.972	2.126	1031.951	0.999

- Output comprises mean, se of mean, sd and various quantiles of parameter values
- Notice the effective samples sizes (n_{eff}):

$$2 \cdot \frac{15 \times 10^3 - 1 \times 10^3}{25} = 1.12 \times 10^3$$

- $\hat{R}$ assesses convergence of chains by comparing "between" and "within" chain variations for a parameter

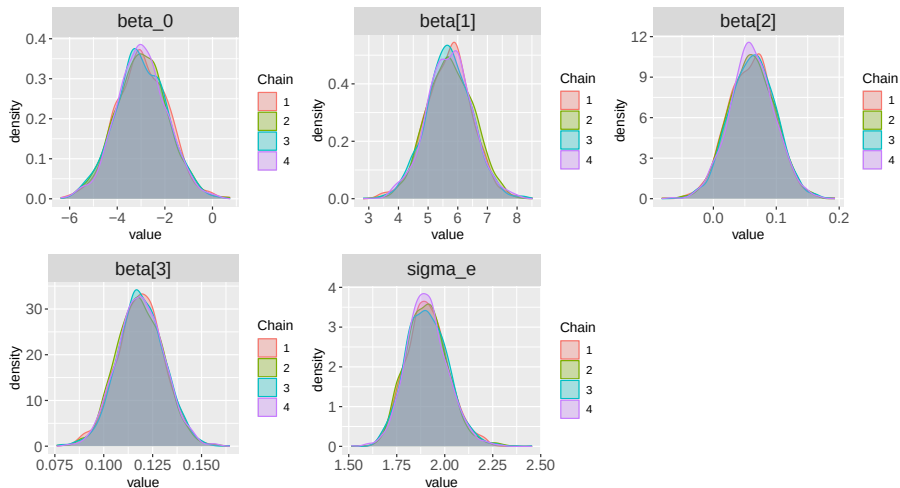
Evaluate (R)STAN model output (posteriors)

```
print(summary(sheep.rs1, pars=c("beta_0", "beta", "sigma_e"))$summary)
```

##		mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
##	beta_0	-3.246	0.035	1.088	-5.320	-3.946	-3.306	-2.534	-1.085	944.527	0.999
##	beta[1]	5.721	0.024	0.747	4.288	5.236	5.728	6.220	7.231	959.127	1.003
##	beta[2]	0.120	0.001	0.036	0.048	0.096	0.119	0.144	0.191	1118.421	1.000
##	beta[3]	0.100	0.000	0.011	0.078	0.093	0.100	0.108	0.121	953.045	0.999
##	sigma_e	1.902	0.003	0.107	1.710	1.828	1.897	1.972	2.126	1031.951	0.999

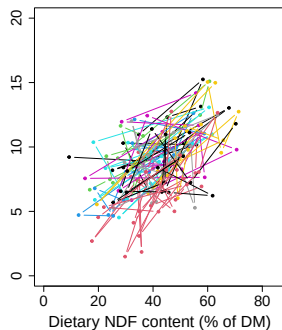
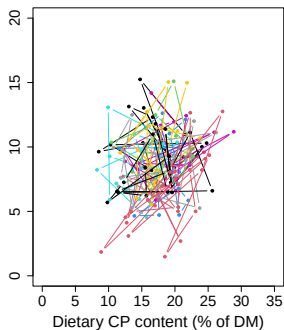
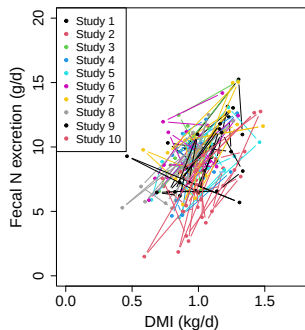
- Output comprises mean, se of mean, sd and various quantiles of parameter values
- Notice the effective samples sizes (n_{eff}):
$$2 \cdot \frac{15 \times 10^3 - 1 \times 10^3}{25} = 1.12 \times 10^3$$
- $\hat{R}$ assesses convergence of chains by comparing "between" and "within" chain variations for a parameter
- $\hat{R} \approx 1.00$, preferably $\hat{R} < 1.01$, certainly $\hat{R} < 1.05$

Model simulation in RSTAN - Posterior density plots



- Satisfied with the chains on these density plots?

Hierarchical or "Mixed-effects" modeling



$$y_{ij} = \beta_0 + \beta_1 x_{1,ij} + \beta_2 x_{2,ij} + \beta_3 x_{3,ij} + s_j + \epsilon_{ij} \quad (8)$$

$$s_j \sim N(0, \sigma_s^2) \quad (9)$$

$$\epsilon_{ij} \sim N(0, \sigma_e^2) \quad (10)$$

Hierarchical linear prediction model code in STAN

```
FN_MixedEf <- "  
  data {  
    int<lower=1> N;           // number of data points  
    real y[N];               // the FN data  
    real x_DMI[N];           // the covariate 1 data  
    real x_CP[N];            // the covariate 2 data  
    real x_NDF[N];           // the covariate 3 data  
    int<lower=1> J;           // number of Studies  
    int<lower=1, upper=J> EXP[N]; // the Studies  
  } // end of data  
  
  parameters{  
    real beta_0;              // regression coefficients  
    real beta[3];             // regression coefficients  
    real u[J];                // experiment intercepts  
    real<lower=0> sigma_u;     // experiment std dev  
    real<lower=0> sigma_e;     // residual std dev  
  } // end of parameters  
  
  model {  
    for(i in 1:N){  
      y[i] ~ normal(beta_0 + beta[1]*x_DMI[i] + beta[2]*x_CP[i] + beta[3]*x_NDF[i] + u[EXP[i]], sigma_e);  
    } // end of for loop i  
  
    beta_0 ~ normal(0,300); beta ~ normal(0,300)  
    u ~ normal(0,sigma_u);  
    sigma_u ~ cauchy(0,15); sigma_e ~ cauchy(0,15);  
  } // end of model  
"
```

Hierarchical linear model RSTAN code

```
FN_MixedEf_mod <- stan_model(model_code=FN_MixedEf,model_name="FN_MixedEf")

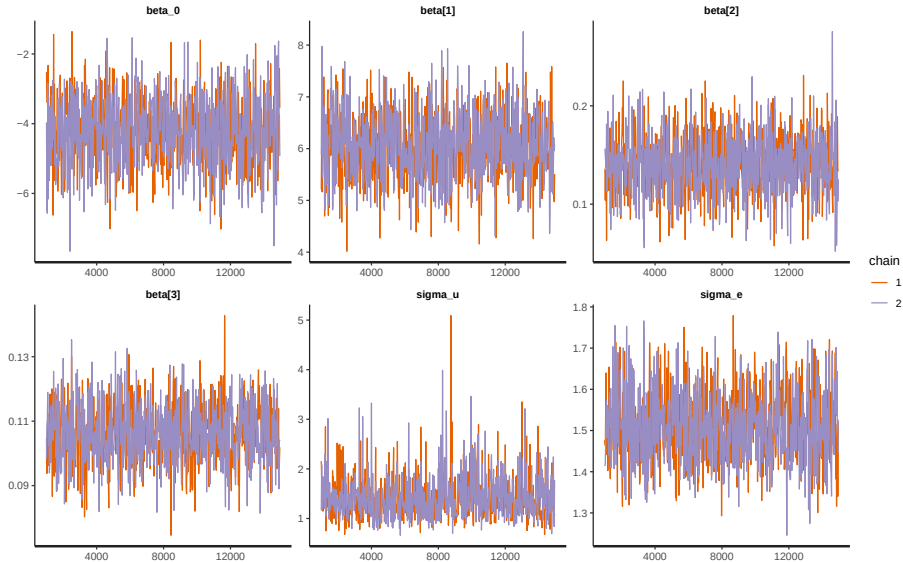
### Extend the list data object with study information
sheep_FN_dat$J <- n_studies
sheep_FN_dat$EXP <- rep(1:n_studies, each=n_sample/n_studies)

### Run the HMC simulation of the STAN model
sheep.rs2 <- sampling(FN_MixedEf_mod, data=sheep_FN_dat,
                      iter=15e3, warmup=1e3, thin=25, chains=2,
                      pars=c("beta_0", "beta", "u", "sigma_u", "sigma_e"))

print(summary(sheep.rs2, pars=c("beta_0", "beta", "u", "sigma_u", "sigma_e"))$summary)
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
##beta_0	-4.222	0.030	1.000	-6.198	-4.892	-4.244	-3.523	-2.349	1079.988	1.000
##beta[1]	6.050	0.019	0.651	4.822	5.621	6.042	6.474	7.324	1161.076	1.000
##beta[2]	0.141	0.001	0.031	0.082	0.120	0.141	0.161	0.202	1300.157	1.001
##beta[3]	0.106	0.000	0.009	0.088	0.100	0.106	0.113	0.124	1103.222	0.999
##u[1]	0.960	0.017	0.573	-0.110	0.570	0.934	1.334	2.157	1091.671	1.000
##u[2]	-1.830	0.017	0.552	-2.937	-2.186	-1.839	-1.451	-0.746	1092.542	1.001
##u[3]	1.485	0.018	0.559	0.373	1.140	1.480	1.827	2.614	1009.265	1.000
. . .										
##u[9]	-0.687	0.018	0.576	-1.823	-1.064	-0.700	-0.312	0.463	1007.436	1.000
##u[10]	-1.994	0.017	0.584	-3.123	-2.388	-1.981	-1.612	-0.846	1174.574	0.999
##sigma_u	1.422	0.015	0.445	0.824	1.120	1.346	1.607	2.533	906.891	1.005
##sigma_e	1.511	0.003	0.089	1.350	1.451	1.505	1.568	1.697	1144.163	1.003

Hierarchical linear model traceplots



Vectorize linear model equation

Model equation in scalar and vectorized form

$$y_{ij} = \beta_0 + \beta_1 x_{1,ij} + \beta_2 x_{2,ij} + \beta_3 x_{3,ij} + s_j + \epsilon_{ij} \quad (11)$$

$$y_{ij} = \mathbf{x}_{ij}^T \boldsymbol{\beta} + s_j + \epsilon_{ij} \quad (12)$$

"Random-effects" of the model

$$s_j \sim N(0, \sigma_s^2) \quad (13)$$

$$\epsilon_{ij} \sim N(0, \sigma_e^2) \quad (14)$$

- Vectorizing in STAN speeds up the MCMC simulation and shortens runtime

Vectorized linear model STAN code

```
FN_MixedEf_X <- "  
  data {  
    int<lower=0> N;           // number of data points  
    real y[N];               // the FN data  
    int<lower=1> K;           // number of predictors  
    matrix[N,K+1] x;         // the N covariate dataframe  
    int<lower=1> J;           // number of experiments  
    int<lower=1, upper=J> EXP[N]; // the experiments  
  } // end of data  
  
  parameters{  
    vector[K+1] beta;        // slope rcs  
    vector[J] u;              // experiment intercepts  
    real<lower=0> sigma_u;    // experiment std dev  
    real<lower=0> sigma_e;    // residual std dev  
  } // end of parameters  
  
  model {  
    y ~ normal(x*beta + u[EXP], sigma_e);  
  
    beta ~ normal(0,300);  
    u ~ normal(0,sigma_u);  
    sigma_u ~ cauchy(0,15);  
    sigma_e ~ cauchy(0,15);  
  } // end of model  
"
```

Vectorized linear model RSTAN code

```
FN_MixedEf_X_mod <- stan_model(model_code=FN_MixedEf_X,model_name="FN_MixedEf_X")

### Extend the list data object with study information
sheep_FN_dat_X <- list(N=sheep_FN_dat$N, y=sheep_FN_dat$y, K=3, J=sheep_FN_dat$J, EXP=sheep_FN_dat$EXP,
                      x=data.frame(x_0=1, DMI=sheep_FN_dat$x_DMI,
                                   CP=sheep_FN_dat$x_CP, NDF=sheep_FN_dat$x_NDF))

# Note first column of data.frame object x
print(head(sheep_FN_dat_X$x))

##      x_0      DMI      CP      NDF
## 1      1 1.099482 21.264705 9.288876
## 2      1 1.135212 19.741308 36.469310
## 3      1 1.219533 13.050128 57.486815
## 4      1 1.229884 16.637352 45.220294
## .      .      .      .

### Run the HMC simulation of the STAN model
sheep.rs3 <- sampling(FN_MixedEf_X_mod, data=sheep_FN_dat_X, pars=c("beta","u","sigma_u","sigma_e"),
                     iter=15e3, warmup=1e3, thin=25, chains=2)

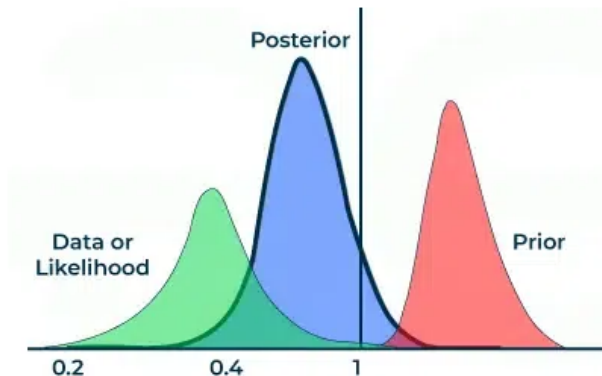
print(summary(sheep.rs3, pars=c("beta","sigma_u","sigma_e"))$summary)

##           mean se_mean   sd    2.5%    25%    50%    75%   97.5%   n_eff   Rhat
##beta[1] -4.166    0.029 1.036 -6.315 -4.830 -4.129 -3.457 -2.174 1237.621 1.000
##beta[2]  6.023    0.020 0.657  4.695  5.610  6.028  6.467  7.292 1084.055 1.000
##beta[3]  0.141    0.001 0.031  0.082  0.120  0.140  0.162  0.201 1072.481 0.999
##beta[4]  0.106    0.000 0.010  0.087  0.100  0.106  0.113  0.125 1099.593 1.001
##sigma_u  1.414    0.015 0.435  0.803  1.118  1.335  1.615  2.550  892.255 0.999
##sigma_e  1.511    0.003 0.088  1.355  1.450  1.506  1.567  1.695  899.006 1.000
```


Priors as STAN code input to avoid recompiling

```
FN_MixedEf_X <- "  
  data {  
    int<lower=0> N;                // number of data points  
    real y[N];                   // the FN data  
    int<lower=1> K;                // number of predictors  
    matrix[N,K+1] x;              // the N covariate dataframe  
    int<lower=1> J;                // number of experiments  
    int<lower=1, upper=J> EXP[N]; // the experiments  
  
    int<lower=1, upper=2> prior_type;  
    real prior_sd_beta; real prior_sigma_u; real prior_sigma_e;  
  } // end of data  
  
  . . .  
  
  model {  
    y ~ normal(x*beta + u[EXP], sigma_e);  
  
    beta ~ normal(0,prior_sd_beta);  
    u ~ normal(0,sigma_u);  
    sigma_u ~ cauchy(0,prior_sigma_u);  
    if (prior_type==1) {sigma_e ~ cauchy(0,prior_sigma_e)};  
    if (prior_type==2) {sigma_e ~ uniform(0,prior_sigma_e)};  
  } // end of model  
  
"
```

Thank you for your attention!



Questions?

Exercise lecture 2

- Take the code from the slides and reproduce the model output
- Increase/decrease the number of iterations and chains. Then check mean and standard deviations of posteriors
- Check the run times of the scalar and vectorized model codes
- Play around with priors, both distributions and parameters, and evaluate your output
- Bonus: extend this univariate model to a bivariate prediction model for methane emissions and N excretion
- Bonus: write a linear model that uses Reproducing Kernel Hilbert Spaces: <https://www.ncbi.nlm.nih.gov/books/NBK583975/> or <https://doi.org/10.1534/genetics.114.164442>